# CONNECT - CAMPUS COLLABORATION & NETWORKING PORTAL

**23IT53C - MODERN WEB TECHNOLOGIES**

**PROJECT REPORT**

*Submitted by*

**INFHANT JOSEPH T 2403957620521052**

*In partial fulfillment for the award of the degree*

*of*

**BACHELOR OF TECHNOLOGY**

*in*

**INFORMATION TECHNOLOGY**




**NATIONAL ENGINEERING COLLEGE**

(An Autonomous Institution affiliated to Anna University, Chennai)

**K.R.NAGAR, KOVILPATTI - 628503**

**OCTOBER - 2025**

# NATIONAL ENGINEERING COLLEGE

(An Autonomous Institution affiliated to Anna University, Chennai)

**K.R.NAGAR, KOVILPATTI - 628503**

## BONAFIDE CERTIFICATE

This is to certify that this project report, **"conNECt - Campus Collaboration & Networking Portal"**, is the bonafide work of **INFHANT JOSEPH T (24205052)** who carried out the project work under my supervision.

**Course Instructor / Guide**

Submitted to the **23IT53C - MODERN WEB TECHNOLOGIES** Viva-Voce examination held at National Engineering College, K.R.Nagar, Kovilpatti on _____

**Internal Examiner** **Co Examiner**

# TABLE OF CONTENTS


| CH NO | TITLE | PAGE NO |
|---|---|---|
| 1 | INTRODUCTION | 3 |
| 2 | OBJECTIVES | 4 |
| 3 | DESCRIPTION | 5 |
| 4 | CLASS DIAGRAM | 6 |
| 5 | TABLE STRUCTURES | 7 |
| 6 | ER DIAGRAM | 11 |
| 7 | TECH STACK | 12 |
| 8 | MODULES | 13 |
| 9 | CONCLUSION | 28 |

# CHAPTER 1

# INTRODUCTION

The **conNECt Campus Collaboration & Networking Portal** is an integrated web-based social and academic platform developed specifically for National Engineering College (NEC), Kovilpatti. It is designed to replace fragmented communication channels and manual academic workflows by creating a unified digital campus ecosystem for students, faculty members, alumni, and platform administrators. Built on the modern MERN stack (MongoDB, Express.js, React.js, and Node.js) paired with Vite and Tailwind CSS, conNECt establishes a secure, high-performance community space that promotes peer networking, cross-batch knowledge sharing, and academic mentorship.

The platform supports four specialized user roles: **Student**, **Faculty**, **Alumni**, and **Admin**, each configured with specific interface layouts, access privileges, and workflows. Students can share technical updates, collaborate on semester projects, join academic subject classrooms, seek guidance through structured help requests, and connect with faculty and alumni mentors. Faculty members can manage dedicated course classrooms, post date-wise lecture summaries, share study materials and assignments, and supervise capstone student projects. Alumni can maintain lasting bonds with their alma mater, share career wisdom, review junior projects, and participate in one-on-one mentorship sessions.

Administrators are provided with a centralized governance dashboard to supervise platform activities, review analytics across academic departments, verify new student and alumni credentials, manage user roles, and moderate feedback. Real-time event notifications and persistent socket-based direct messaging keep the entire campus community seamlessly connected, while JWT-based role authorization guarantees data protection across all API endpoints.

The conNECt portal significantly improves student engagement, bridges the gap between active learners and seasoned alumni in the industry, and simplifies academic administrative overhead. Its responsive, glassmorphic user interface delivers an intuitive experience across mobile devices, tablets, and desktop computers, providing National Engineering College with a future-proof collaborative network.

# CHAPTER 2

# OBJECTIVES

- To provide robust role-based authentication and authorization (RBAC) ensuring secure and tailored access for Students, Faculty, Alumni, and Administrators.
- To maintain a centralized MongoDB database for storing user credentials, academic classrooms, student projects, mentorship requests, real-time chats, and notifications.
- To develop an interactive Campus Feed that allows students, faculty, and alumni to post technical updates, share research achievements, upload media, and engage via comments and likes.
- To create dedicated Academic Classrooms where faculty members can publish date-wise lecture summaries, study materials, announcements, and track assignment tasks.
- To implement an end-to-end Project Incubator module enabling students to propose capstone ideas, form multidisciplinary teams, track development stages, and request faculty or alumni mentors.
- To establish a formal Mentorship Lifecycle featuring duration-based mentor requests, accept/reject workflows, and dedicated collaboration workspaces with Google Meet integrations.
- To develop a real-time Direct Messaging and communication facility using WebSockets to support instant peer-to-peer dialogues, faculty clarifications, and alumni career guidance.
- To build an Academic Help Request system where students can broadcast targeted questions regarding programming, database queries, and lab exercises to peer helpers.
- To facilitate Alumni Transition & Networking, allowing graduates to showcase their corporate roles, share industry insights, and preserve college memories.
- To deliver an Administrative Management Dashboard for managing user verification status, monitoring department activity metrics, and moderating community suggestions.
- To implement secure credential handling, bcrypt password hashing, JWT token verification, and React Protected Routes to prevent unauthorized endpoint access.
- To provide a responsive, aesthetic user interface using Tailwind CSS and Vite that guarantees fast loading times, clean micro-interactions, and accessibility across devices.

# CHAPTER 3

# DESCRIPTION

The conNECt Campus Collaboration & Networking Portal operates as a centralized web service built to bridge the gap between academic learning and professional networking within National Engineering College. Traditional institutional portals often treat learning management and social interaction as completely disconnected utilities. conNECt integrates both aspects into a single, cohesive web platform where curricular activities, project incubations, and career mentorship reinforce one another naturally.

Upon accessing the portal, users encounter a role-aware authentication page where they can sign in as Student, Faculty, or Alumni using institutional credentials (@nec.edu.in) or verified emails. Once authenticated, the system evaluates their role in the database and renders the appropriate dashboard layout. Students are greeted with an overview of enrolled classes, ongoing project collaborations, and trending campus discussions. Faculty members access course management tools, student rosters, and incoming mentorship requests. Alumni are provided with corporate collaboration channels, junior mentorship queues, and college memories.

The project collaboration module allows students to publish capstone ideas, specify required tech stacks (e.g., React, Node.js, Python), and invite teammates. Once formed, the team can submit formal mentorship requests to faculty or alumni specialists. Mentors can review project goals, accept requests with customized durations (e.g., 30, 60 days), and collaborate within shared workspaces equipped with progress trackers and discussion threads.

Data security and integrity are enforced across every transaction. The backend API is constructed using Express.js and modular MVC architecture, utilizing Mongoose ODM to enforce schema validation and referential integrity across MongoDB collections. Sensitive operations such as user verification and role modification are strictly restricted to authenticated platform administrators. Real-time event notifications ensure users are immediately alerted when connection requests are accepted, new classroom assignments are posted, or mentorship queries are answered.

# CHAPTER 4

# CLASS DIAGRAM

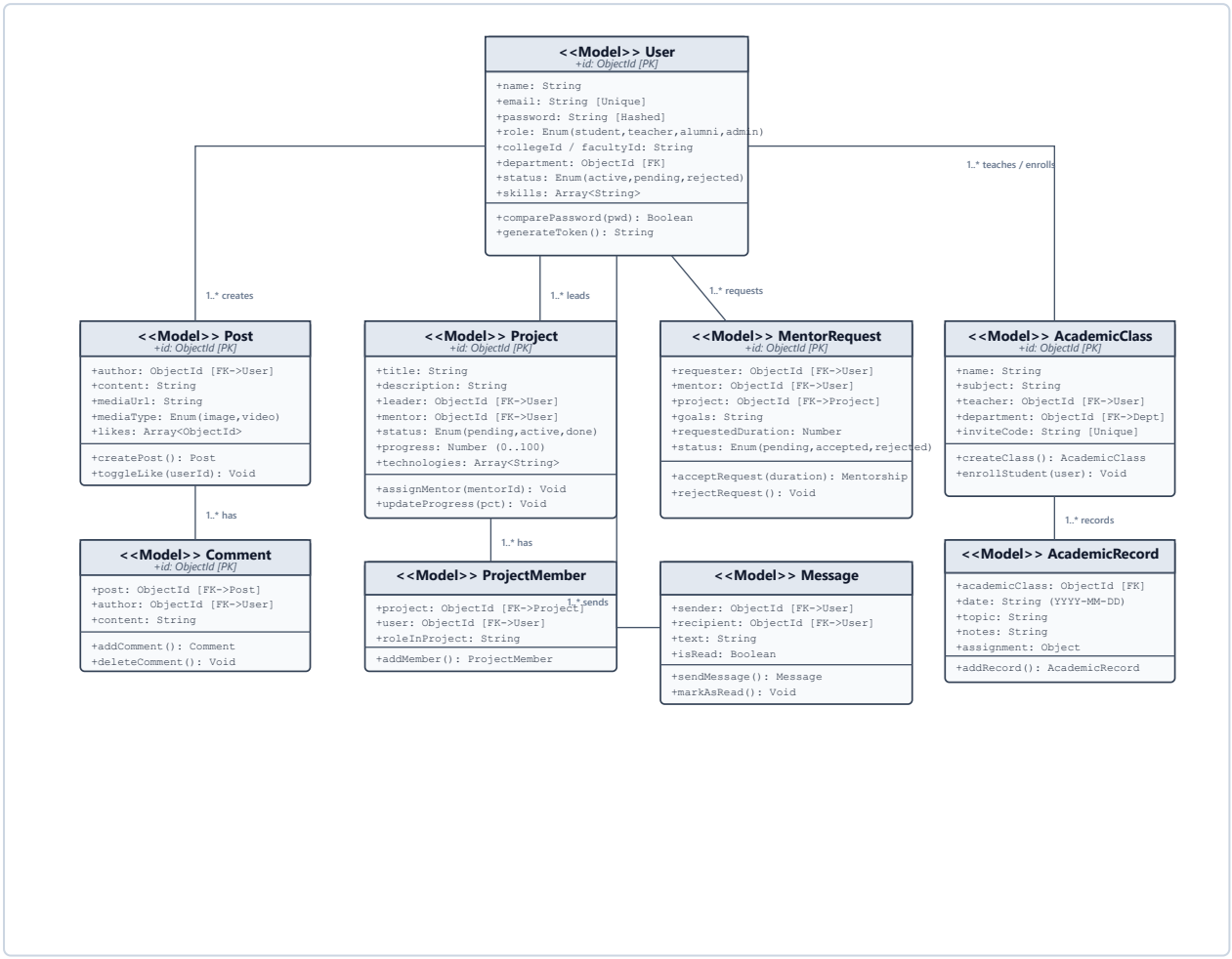



**Fig 4.1 - UML Class Diagram of conNECt Portal Entities**

# CHAPTER 5

# TABLE STRUCTURES

## 1. users Collection

*Stores authentication credentials, profile information, and role classifications for all members.*

| Field | Type | Null | Key | Default | Description |
|---|---|---|---|---|---|
| `_id` | ObjectId | NO | PRI | Auto | Primary key identifier |
| `name` | String | NO | | None | Full name of the user |
| `email` | String | NO | UNI | None | Normalized primary email |
| `password` | String | NO | | None | Bcrypt hashed password |
| `role` | String | NO | | 'student' | student / teacher / alumni / admin |
| `collegeId` | String | YES | UNI | None | Student roll/college ID |
| `facultyId` | String | YES | UNI | None | Faculty staff ID code |
| `department` | ObjectId | YES | MUL | None | Ref -> Department |
| `year` | Number | YES | | None | Current year of study (1 to 4) |
| `status` | String | NO | | 'active' | active / pending_approval / rejected |
| `skills` | Array | YES | | [] | Technical competencies list |
| `emailVerified` | Boolean | NO | | false | Email verification flag |

## 2. departments Collection

*Manages college engineering departments and metadata.*

| Field | Type | Null | Key | Default | Description |
|---|---|---|---|---|---|
| `_id` | ObjectId | NO | PRI | Auto | Department primary key |
| `name` | String | NO | | None | Full department title (e.g. IT, CSE) |
| `code` | String | NO | UNI | None | Short code (e.g., 'IT', 'ECE') |
| `themeColor` | String | YES | | None | Display brand accent hex code |

## 3. posts Collection

*Contains campus feed publications, media attachments, and engagement statistics.*

| Field | Type | Null | Key | Default | Description |
|---|---|---|---|---|---|
| `_id` | ObjectId | NO | PRI | Auto | Post unique identifier |
| `author` | ObjectId | NO | MUL | None | Ref -> User (Creator) |
| `content` | String | YES | | None | Post narrative / description text |
| `mediaUrl` | String | YES | | None | Uploaded image/video asset link |

### 4. comments Collection

*Stores conversational replies and discussions linked to specific campus feed posts.*

| Field | Type | Null | Key | Default | Description |
|---|---|---|---|---|---|
| `_id` | ObjectId | NO | PRI | Auto | Comment unique ID |
| `post` | ObjectId | NO | MUL | None | Ref -> Post |
| `author` | ObjectId | NO | MUL | None | Ref -> User (Author) |
| `content` | String | NO | | None | Text of comment discussion |

### 5. projects Collection

*Maintains capstone, hackathon, and research project repositories.*

| Field | Type | Null | Key | Default | Description |
|---|---|---|---|---|---|
| `_id` | ObjectId | NO | PRI | Auto | Project primary key |
| `title` | String | NO | | None | Project title name |
| `description` | String | YES | | None | Detailed technical specification |
| `leader` | ObjectId | NO | MUL | None | Ref -> User (Project Owner) |
| `mentor` | ObjectId | YES | MUL | null | Ref -> User (Faculty/Alumni) |
| `technologies` | Array | YES | | [] | Tags (e.g. React, Node.js, AI) |
| `status` | String | NO | | 'pending' | pending / in_progress / completed |
| `progress` | Number | NO | | 0 | Progress completion percentage |
| `githubLink` | String | YES | | None | Repository URL link |

### 6. project_members Collection

*Maps student collaborators and role assignments to active projects.*

| Field | Type | Null | Key | Default | Description |
|---|---|---|---|---|---|
| `_id` | ObjectId | NO | PRI | Auto | Membership record ID |
| `project` | ObjectId | NO | MUL | None | Ref -> Project |
| `user` | ObjectId | NO | MUL | None | Ref -> User (Collaborator) |
| `roleInProject` | String | NO | | 'Member' | Role title in the project |

### 7. academic_classes Collection

*Defines curriculum classrooms managed by professors.*

| Field | Type | Null | Key | Default | Description |
|---|---|---|---|---|---|
| `_id` | ObjectId | NO | PRI | Auto | Class unique identifier |
| `name` | String | NO | | None | Course title and code (e.g. IT301) |

### 8. academic_records Collection

*Stores date-wise lecture summaries, study materials, announcements, and assignments.*

| Field | Type | Null | Key | Default | Description |
|---|---|---|---|---|---|
| `_id` | ObjectId | NO | PRI | Auto | Record unique key |
| `academicClass` | ObjectId | NO | MUL | None | Ref -> AcademicClass |
| `date` | String | NO | | None | Formatted calendar date (YYYY-MM-DD) |
| `creator` | ObjectId | NO | MUL | None | Ref -> User (Posting Teacher) |
| `topic` | String | YES | | None | Lecture unit/topic title |
| `notes` | String | YES | | None | Class notes & concept summary |
| `announcement` | String | YES | | None | Urgent course notice / lab dates |
| `assignment` | Object | YES | | null | Title, description, and dueDate |

### 9. mentor_requests Collection

*Logs student applications submitted to faculty and alumni mentors.*

| Field | Type | Null | Key | Default | Description |
|---|---|---|---|---|---|
| `_id` | ObjectId | NO | PRI | Auto | Request unique ID |
| `requester` | ObjectId | NO | MUL | None | Ref -> User (Student Leader) |
| `mentor` | ObjectId | NO | MUL | None | Ref -> User (Faculty/Alumni) |
| `project` | ObjectId | YES | MUL | None | Ref -> Project |
| `requestType` | String | NO | | 'individual' | 'individual' or 'group' |
| `goals` | String | NO | | None | Specific mentorship guidance needs |
| `requestedDuration` | Number | NO | | 30 | Requested mentorship days |
| `status` | String | NO | | 'pending' | pending / accepted / rejected / cancelled |

### 10. mentorships Collection

*Maintains approved active mentorship collaborations and timeline durations.*

| Field | Type | Null | Key | Default | Description |
|---|---|---|---|---|---|
| `_id` | ObjectId | NO | PRI | Auto | Mentorship engagement ID |
| `mentor` | ObjectId | NO | MUL | None | Ref -> User (Mentor) |
| `mentee` | ObjectId | YES | MUL | None | Ref -> User (Student Mentee) |
| `project` | ObjectId | YES | MUL | None | Ref -> Project |
| `request` | ObjectId | NO | UNI | None | Ref -> MentorRequest |
| `startDate` | Date | NO | | Now | Mentorship start timestamp |
| `endDate` | Date | NO | | None | Mentorship expiration deadline |

### 11. messages Collection

*Records one-on-one direct communication between community members.*

| Field | Type | Null | Key | Default | Description |
|---|---|---|---|---|---|
| `_id` | ObjectId | NO | PRI | Auto | Message unique identifier |
| `sender` | ObjectId | NO | MUL | None | Ref -> User (Sender) |
| `recipient` | ObjectId | NO | MUL | None | Ref -> User (Recipient) |
| `text` | String | YES | | None | Direct chat message content |
| `mediaUrl` | String | YES | | None | Uploaded document or image attachment |
| `isRead` | Boolean | NO | | false | Read receipt flag |

### 12. notifications Collection

*Logs real-time alerts and activity updates dispatched to user accounts.*

| Field | Type | Null | Key | Default | Description |
|---|---|---|---|---|---|
| `_id` | ObjectId | NO | PRI | Auto | Notification unique key |
| `user` | ObjectId | NO | MUL | None | Ref -> User (Recipient) |
| `type` | String | NO | | None | mentor_request, comment, like, message... |
| `title` | String | NO | | None | Brief notification summary |
| `message` | String | NO | | None | Full notification body message |
| `relatedId` | ObjectId | YES | | None | Target entity ID for quick routing |
| `isRead` | Boolean | NO | | false | Flag indicating if user saw alert |

### 13. help_requests Collection

*Enables students to broadcast peer academic questions and receive assistance.*

| Field | Type | Null | Key | Default | Description |
|---|---|---|---|---|---|
| `_id` | ObjectId | NO | PRI | Auto | Help request primary key |
| `requester` | ObjectId | NO | MUL | None | Ref -> User (Asking Student) |
| `title` | String | NO | | None | Brief query topic title |
| `subject` | String | NO | | None | Academic subject (e.g., DBMS, Web Tech) |
| `status` | String | NO | | 'open' | open / in_progress / completed |
| `acceptedHelper` | ObjectId | YES | MUL | null | Ref -> User (Peer Helper) |

# CHAPTER 6

# ER DIAGRAM

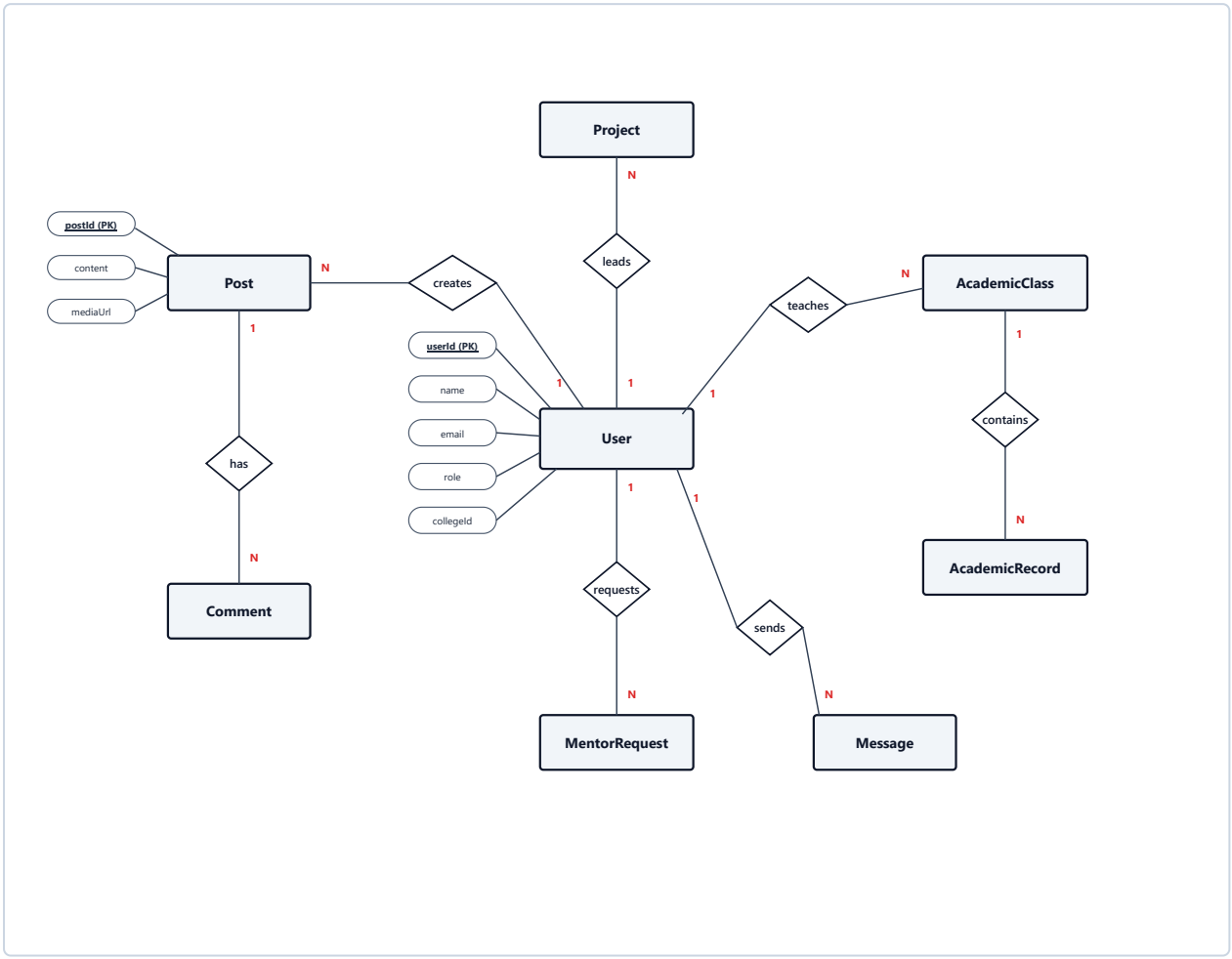



**Fig 6.1 - Entity-Relationship (ER) Diagram of conNECt Database**

# CHAPTER 7

# TECH STACK

**Frontend Technologies:** React.js 18, Vite 5, Tailwind CSS, JavaScript ES6+, HTML5, CSS3

**Backend Technologies:** Node.js, Express.js (RESTful API Server)

**Database:** MongoDB Community Server, Mongoose ODM (Object Data Modeling)

**Real-Time Communication:** Socket.io (Bi-directional WebSocket engine for chat & live notifications)

**Authentication & Security:** JSON Web Tokens (JWT), bcryptjs (Salted Password Hashing), Role-Based Access Control (RBAC)

**Asset Storage & Uploads:** Multer (Multipart Form Data handling), Cloudinary API

**Mail Notifications:** Nodemailer with SMTP transport for email verification and transactional alerts

**Client Libraries:** Axios (HTTP client with request interceptors), React Router DOM v6, Lucide React & React Icons

**Approach:** MVC (Model-View-Controller) Pattern, Component-Driven UI Architecture, Async/Await Non-Blocking I/O

**Architecture: Component-Driven MVC (Model-View-Controller)**

- **Models:** Mongoose schemas defining strict constraints, data types, indexes, and relationship references for User, Project, AcademicClass, Mentorship, Message, and Notification.
- **Views:** Modular React components utilizing functional paradigms, custom hooks, and Tailwind CSS for responsive styling, rendering user portals, dashboards, and modal workflows.
- **Controllers:** Asynchronous Express controller functions handling business logic, validation, aggregation pipelines, and consistent JSON responses.
- **Routes:** Express routers mapping RESTful endpoints (GET, POST, PUT, DELETE, PATCH) protected by JWT verification middleware.
- **Database Layer:** MongoDB document store optimized with compound indexes for high-speed queries on connections, messages, and course memberships.
- **Services:** Standalone backend modules orchestrating socket dispatching, email generation, and automated database cleanup routines.

# CHAPTER 8

# MODULES

## 8.1 FOLDER STRUCTURE

### BACKEND FOLDER STRUCTURE:




**Fig 8.1 - Backend Folder Structure**

The conNECt backend folder structure organizes the server application into dedicated directories for controllers, models, routes, middleware, config, and utility services. Routes define API contract endpoints, controllers execute business logic and database queries, and models enforce MongoDB schemas using Mongoose. This clean modular structure facilitates debugging, independent testing, and high maintainability.

### BACKEND ROUTING & SERVICES:




**Fig 8.2 - Backend Routes & Controller Organization**

In the conNECt portal, routes and controllers form the backbone of the server architecture. Built with Express.js and Node.js, the backend exposes RESTful endpoints for user authentication, campus feed posts, project collaborations, academic classrooms, and administrator controls. Each route module isolates a functional domain:

- `/api/auth`: Handles user registration, JWT login, credential verification, and account password resets.
- `/api/posts`: Coordinates campus feed retrieval, text and media uploads, comments, and post likes.
- `/api/projects`: Manages capstone project incubations, member assignments, and progress milestones.
- `/api/mentorship`: Coordinates formal mentor requests, review workflows, and active collaboration spaces.
- `/api/academic`: Delivers classroom join operations, date-wise lecture notes, and assignments.
- `/api/admin`: Houses administrator governance tools, user verification toggles, and platform metrics.

## FRONTEND FOLDER STRUCTURE:




**Fig 8.3 - Frontend Folder Structure**

The conNECt client is developed as a high-performance Single-Page Application (SPA) using React.js 18 and Vite. The codebase is organized into atomic UI components, responsive layout wrappers, role-specific pages, global authentication contexts, and centralized Axios API services. This component-driven structure guarantees consistent visual styling, reusable design tokens, and optimal bundle size.

## 8.2 ARCHITECTURAL SEPARATION OF CONCERNS

The conNECt application implements a strict separation of concerns across both frontend and backend layers to promote high cohesion and low coupling:

- **API Routing Layer:** Maps incoming HTTP requests to dedicated controllers while applying authentication and validation middleware. Ensures clean URLs and standardized REST HTTP status codes (200, 201, 400, 401, 403, 404, 500).
- **Business Logic (Controllers):** Encapsulates all domain rules, such as verifying project leadership before submitting mentor requests, calculating active mentorship duration dates, and aggregating department engagement.
- **Data Layer (Mongoose ODM):** Manages MongoDB connectivity, enforces data integrity rules, handles auto-indexing, and provides pre-save hooks for password encryption.
- **Global State & Context:** Utilizes React Context (`AuthContext`, `BadgeContext`) to persist authenticated user sessions, JWT tokens, unread message badges, and active portal settings across route transitions without prop drilling.
- **Client API Service Layer:** Centralizes all network requests inside an Axios service instance equipped with automatic token interceptors and unified error handlers.
- **Protected Client Routing:** Wraps sensitive user screens inside `ProtectedRoute` guards that evaluate role entitlements before mounting components, redirecting unauthorized users.

## 8.3 APPLICATION MODULES & SCREEN WALKTHROUGHS

### 1. Login & Role-Aware Authentication:

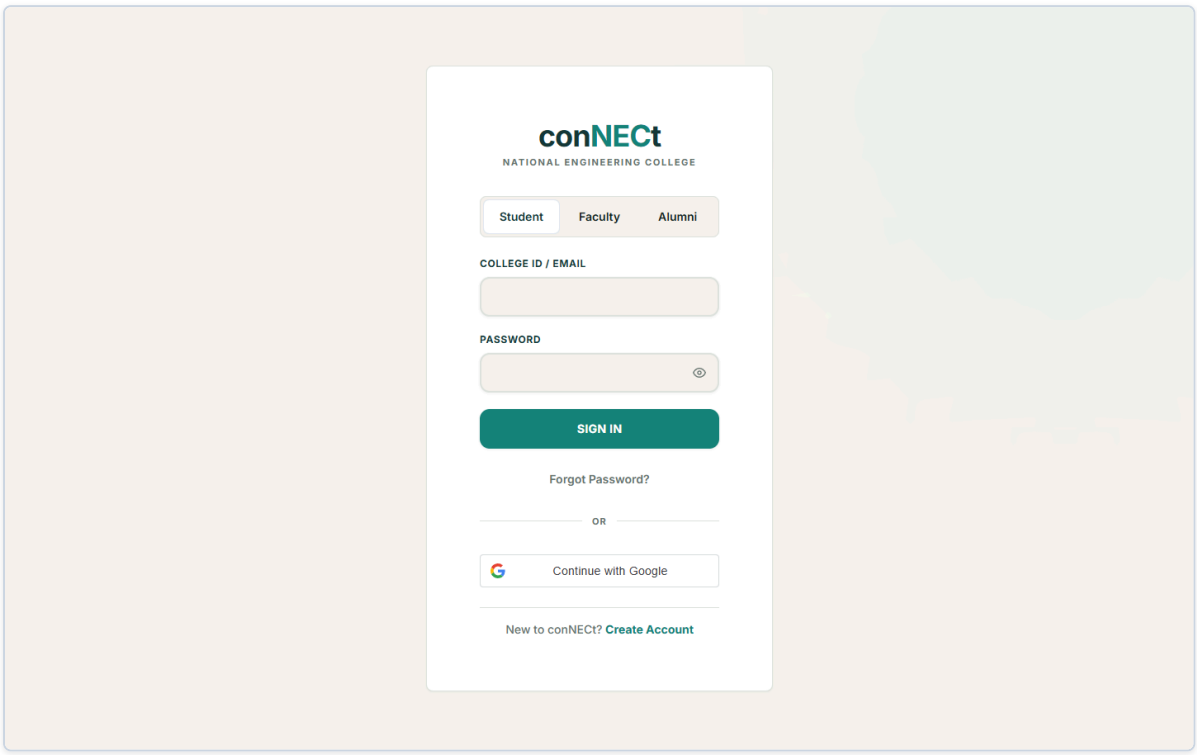



**Fig 8.4 - User Authentication & Role Tab Selection Page**

The authentication page provides a secure access gateway for students, faculty, and alumni. Users can select their role tab (Student, Faculty, Alumni) or authenticate with their institutional email (@nec.edu.in) and password. The system utilizes bcrypt password verification on the backend and issues signed JWT tokens upon success. The interface features a clean, responsive card with password visibility toggling, error alerts, and Google OAuth integration.

## 2. Student Portal & Centralized Dashboard:

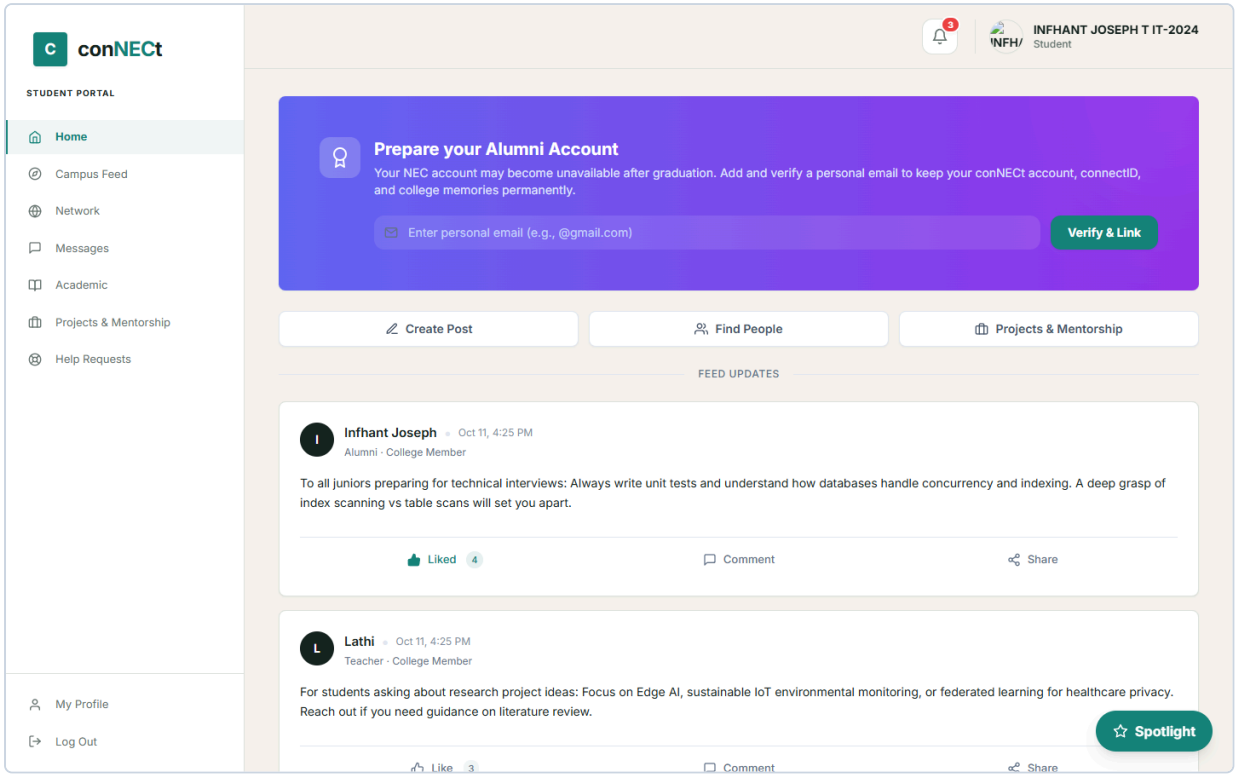



**Fig 8.5 - Student Dashboard Displaying Quick Actions, Recent Items & Feed**

The Student Dashboard serves as the central command center for learners. It displays a personalized welcome greeting, quick-access action shortcuts (Create Post, Projects, Academic Classes, Network), and a recently accessed widgets container. The sidebar navigation provides immediate routing to core modules, while the embedded feed renders real-time college updates and department announcements.

### 3. Alumni Portal & College Legacy:

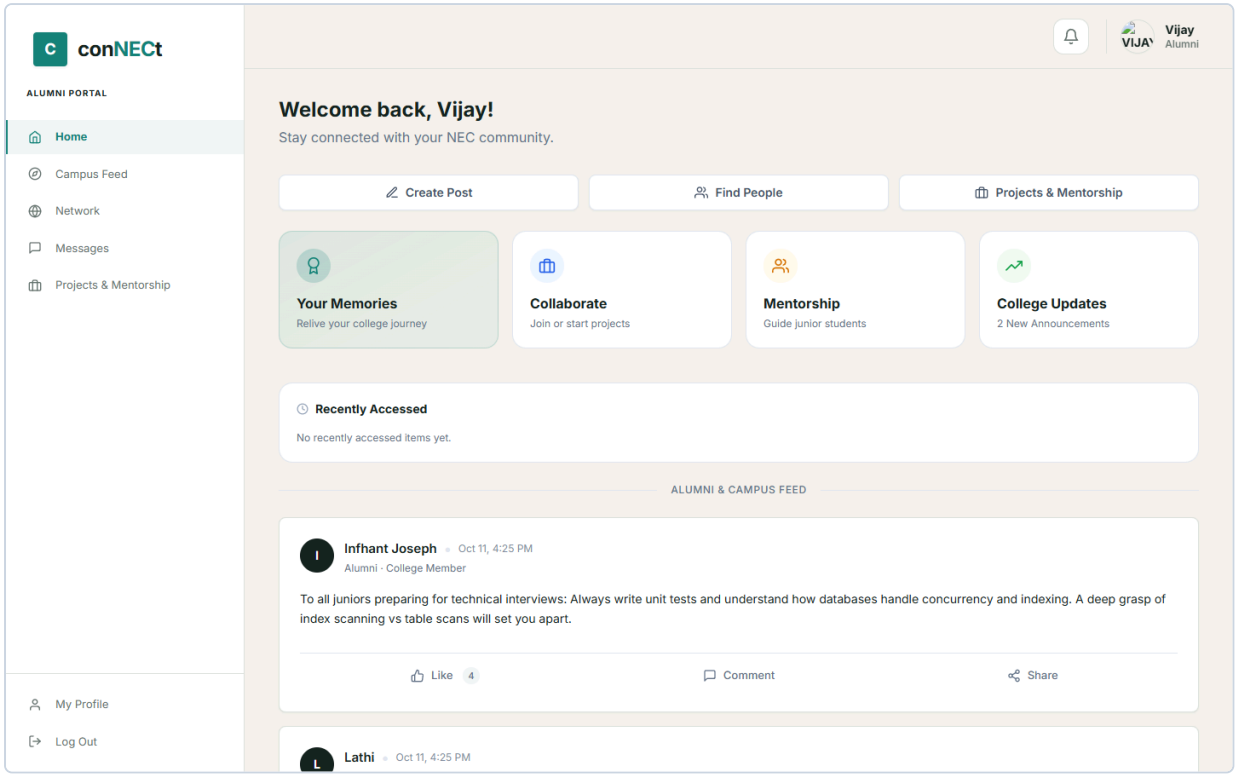



**Fig 8.6 - Alumni Portal Featuring College Memories & Mentorship Shortcuts**

The Alumni Portal provides graduated seniors with dedicated tools to stay engaged with National Engineering College. Alumni can review their historical college memories, browse capstone project collaboration requests, guide junior student innovators, and review campus announcements. The layout highlights corporate designations (e.g., Senior Software Engineer at Zoho, Full Stack Engineer at Freshworks) to inspire current students.

### 4. Campus Feed & Interactive Discussions:

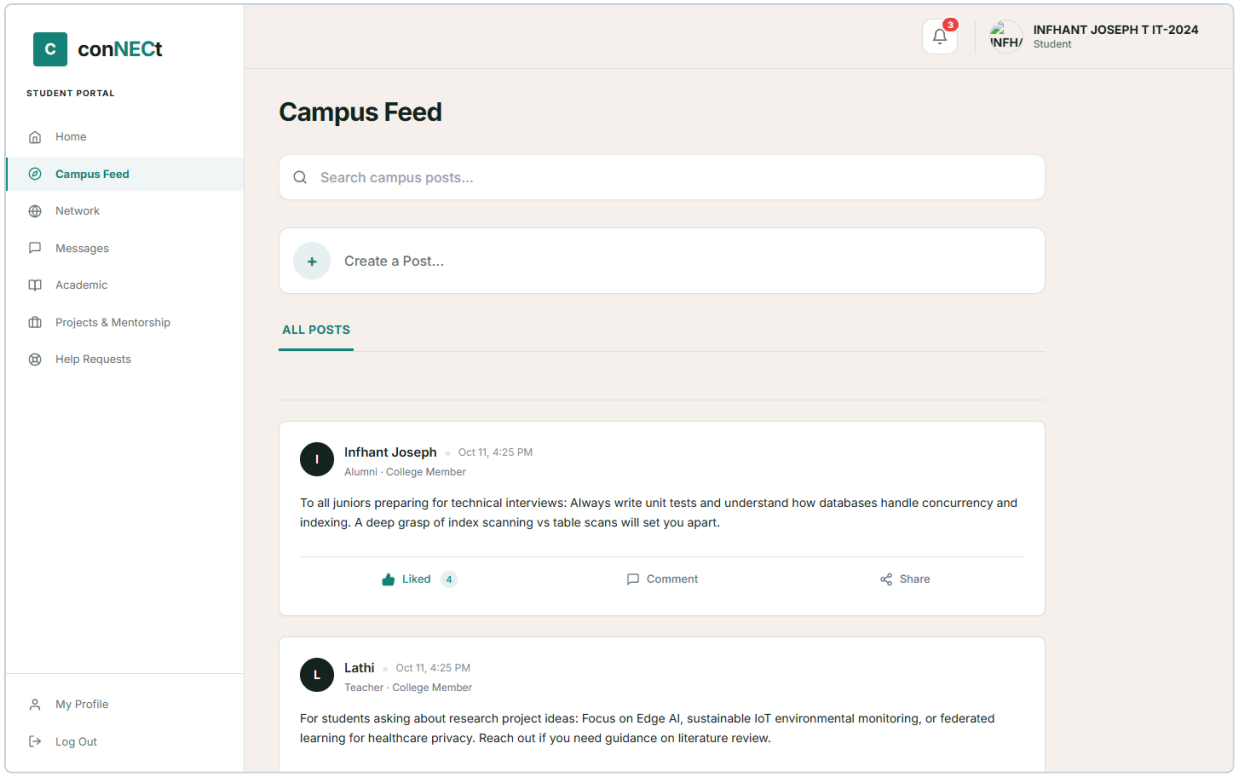



**Fig 8.7 - Campus Feed Showing Peer Posts, Comments & Academic News**

The Campus Feed delivers an interactive timeline of collegiate life. Users can author formatted posts with optional image or video attachments, discuss programming queries, celebrate symposium wins, and review department circulars. Each post supports real-time like toggles and threaded comments where faculty mentors, peers, and alumni actively participate in academic discussions.

### 5. Network & Peer Connections:

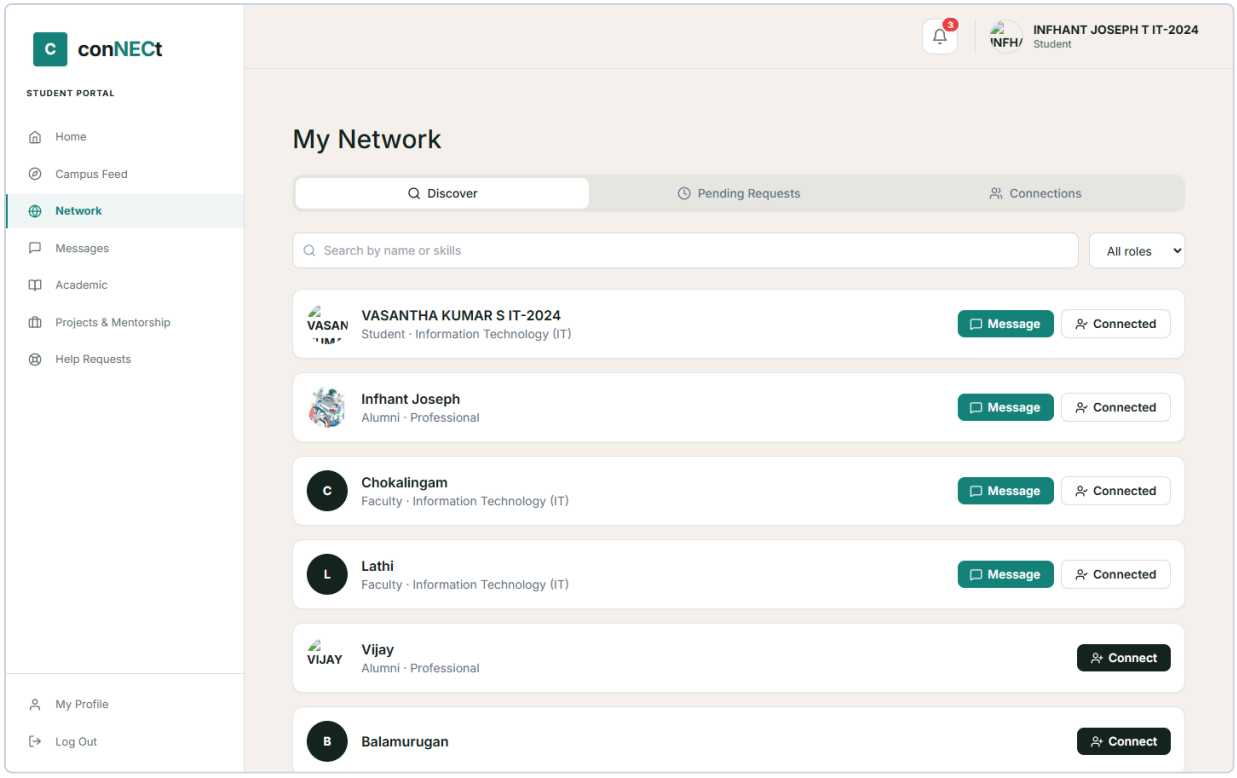



**Fig 8.8 - Campus Network Directory with Filters & Connection Requests**

The Network module facilitates cross-departmental connections between students, faculty, and alumni. Users can discover colleagues filtered by department (IT, CSE, ECE, EEE, AI, Mech) and role, send connection invitations, review incoming pending requests, and browse connected peers. This directory breaks down batch barriers and fosters multidisciplinary project teams.

## 6. Projects Incubator & Mentorship Hub:

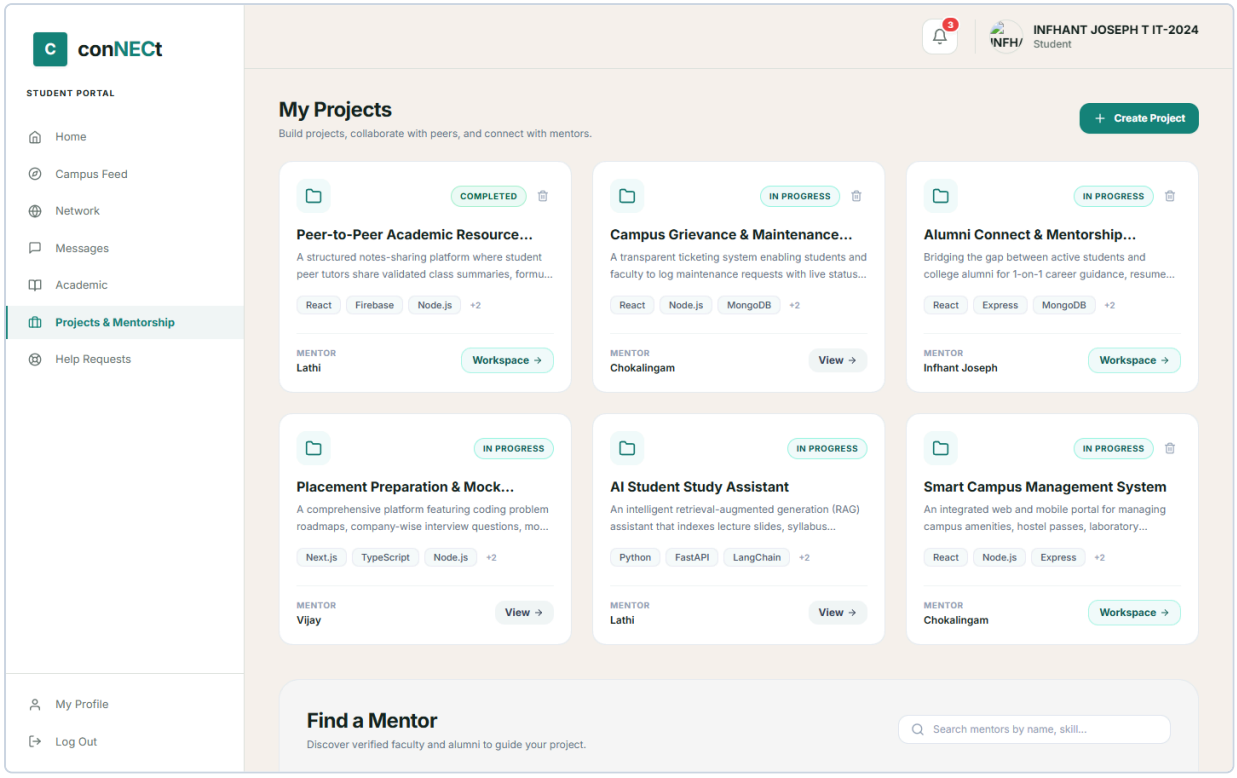



**Fig 8.9 - Projects & Mentorship Hub with Mentor Search & Workspace Access**

The Mentorship Hub serves as an incubator for capstone projects and hackathon teams. Students can showcase projects with technology tags and progress metrics, search for eligible faculty or alumni mentors by technical skill, and submit formal mentorship requests. Mentors receive incoming requests with project goals, which they can accept to launch a collaborative workspace.

## 7. Academic Classrooms & Courseware Workspaces:

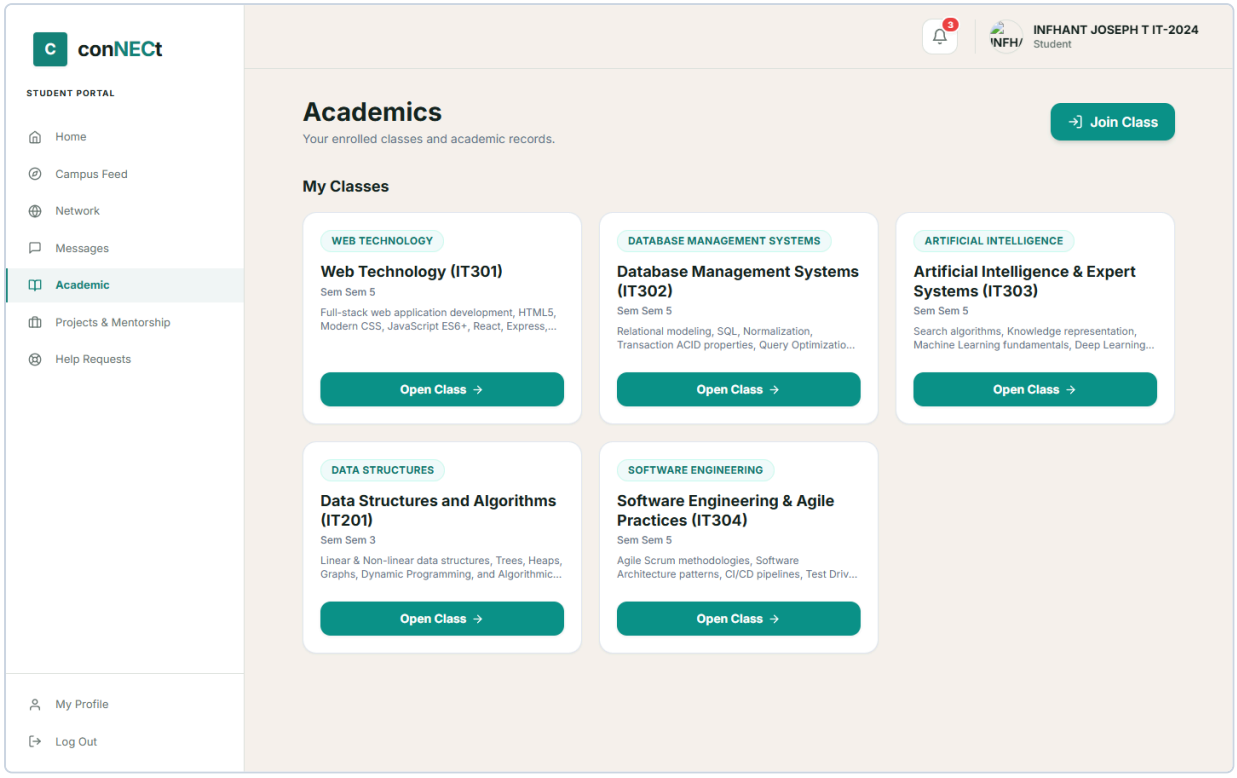



**Fig 8.10 - Academic Dashboard Showing Enrolled Subjects & Lecture Materials**

The Academic module provides structured course workspaces for enrolled subjects (e.g., Web Technology, Database Systems, Artificial Intelligence). Faculty members can publish date-wise lecture topics, downloadable notes, lab schedules, and assignments with due dates. Students can navigate lecture calendars, review classroom announcements, and download materials directly.

### 8. Real-Time Peer & Faculty Direct Messaging:

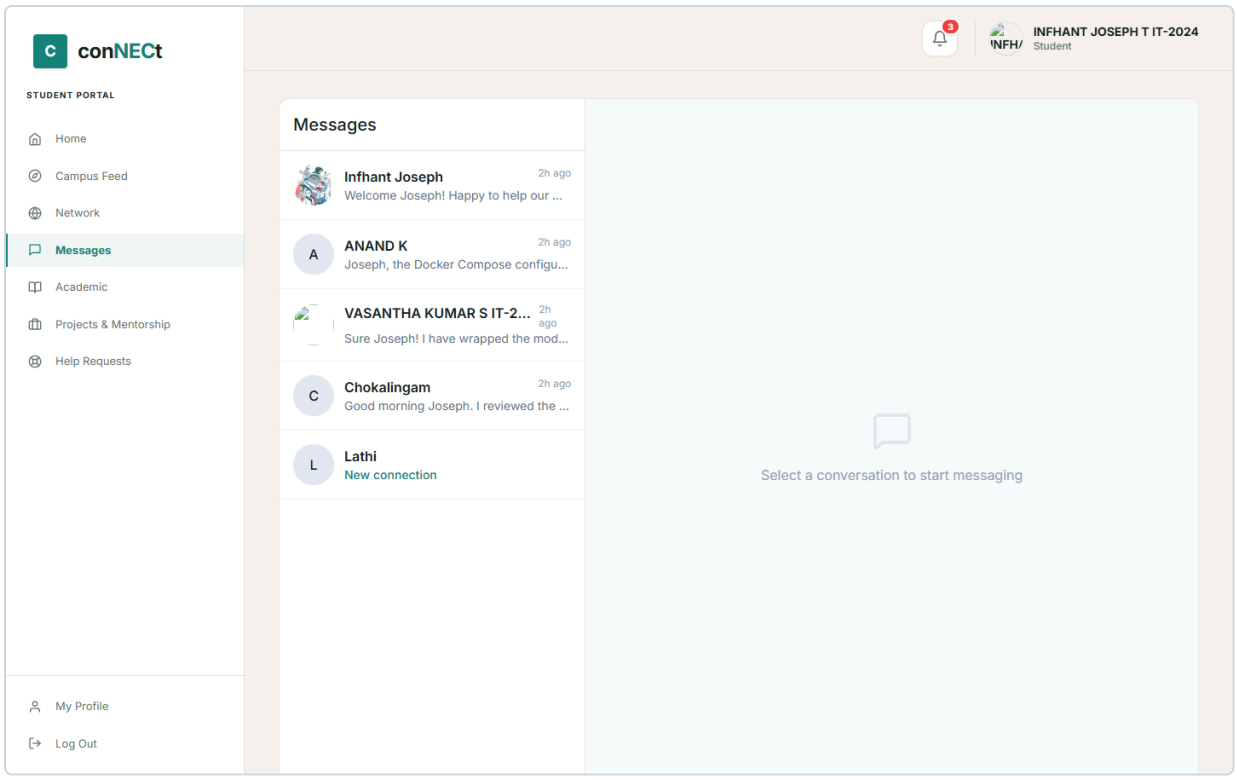



**Fig 8.11 - Real-Time Messaging Interface with Active Conversation Threads**

The Messages module delivers real-time two-way communication between students, faculty mentors, and alumni. Integrated with WebSockets via Socket.io, messages and file attachments appear instantly without page refreshing. Conversation lists display unread message counters, user avatars, and online presence indicators, enabling rapid collaboration on coursework and project hurdles.

### 9. Live Activity Updates & Notifications:

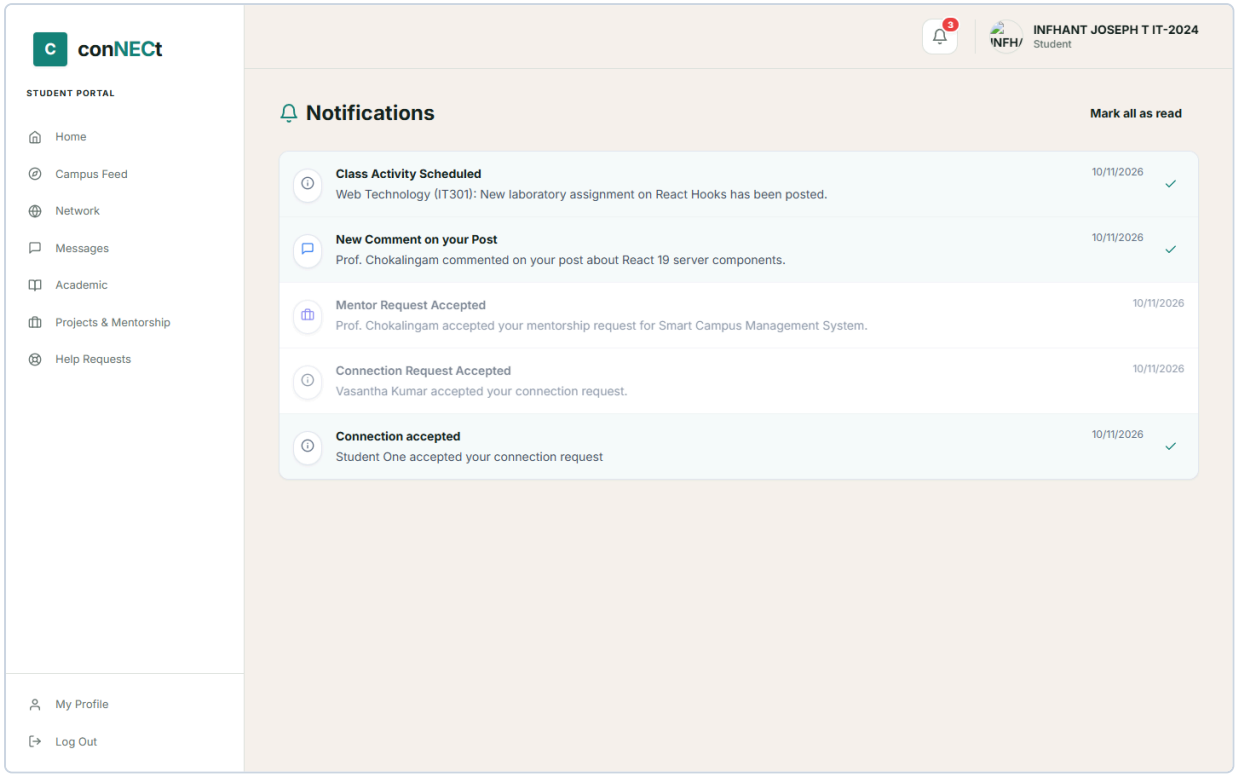



**Fig 8.12 - Notifications Page Displaying Real-Time Platform Event Alerts**

The Notifications center ensures users never miss critical college updates. Event triggers generate automated alerts whenever someone accepts a connection request, comments on a post, publishes a new classroom assignment, or accepts a project mentorship query. Notifications include direct routing links to navigate immediately to the corresponding project or post.

### 10. Comprehensive User Profile & Achievements:

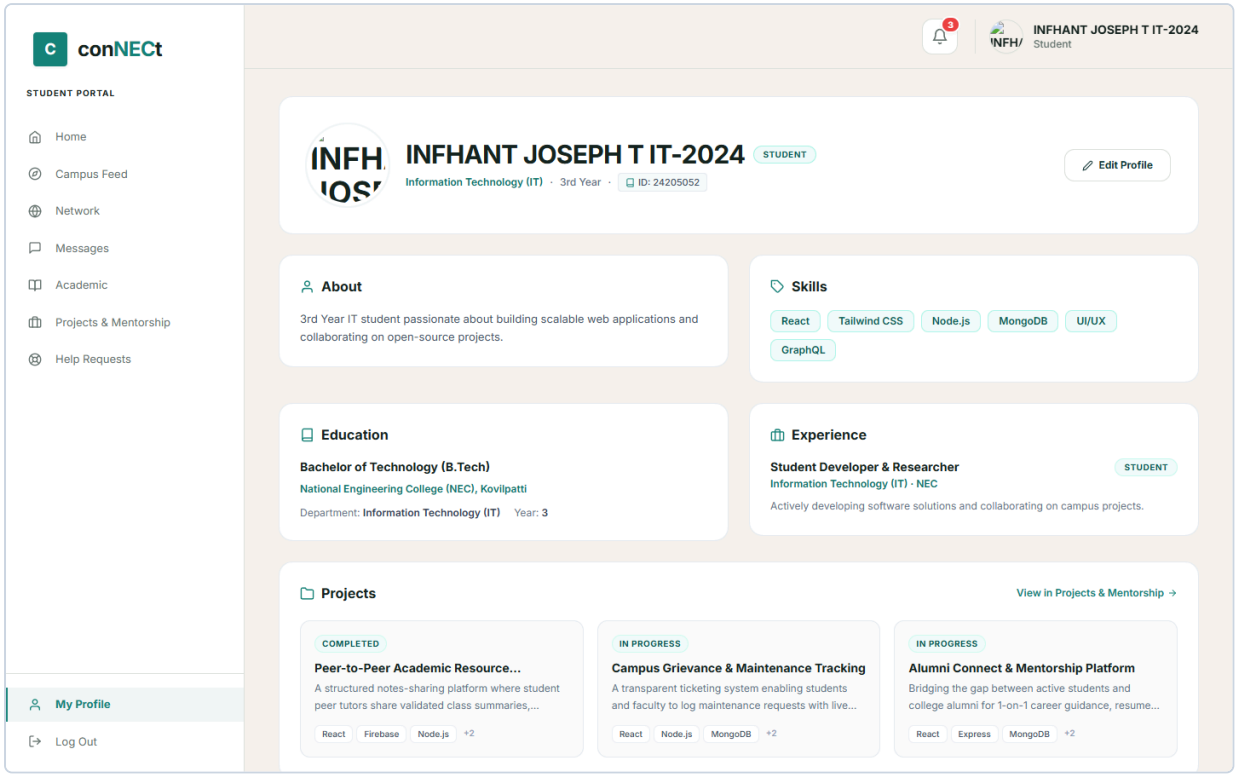



**Fig 8.13 - User Profile Showcase Displaying Skills, Education, Projects & Badges**

The Profile page acts as a digital resume and portfolio for students, faculty, and alumni. It showcases technical skills, verified college IDs, academic department, bio, social links (GitHub, LinkedIn), active project portfolios, and verified achievements (hackathon wins, research papers, AWS certifications). Users can easily update their profile details through a modal editing form.

## 11. Administrator Governance & User Verification:

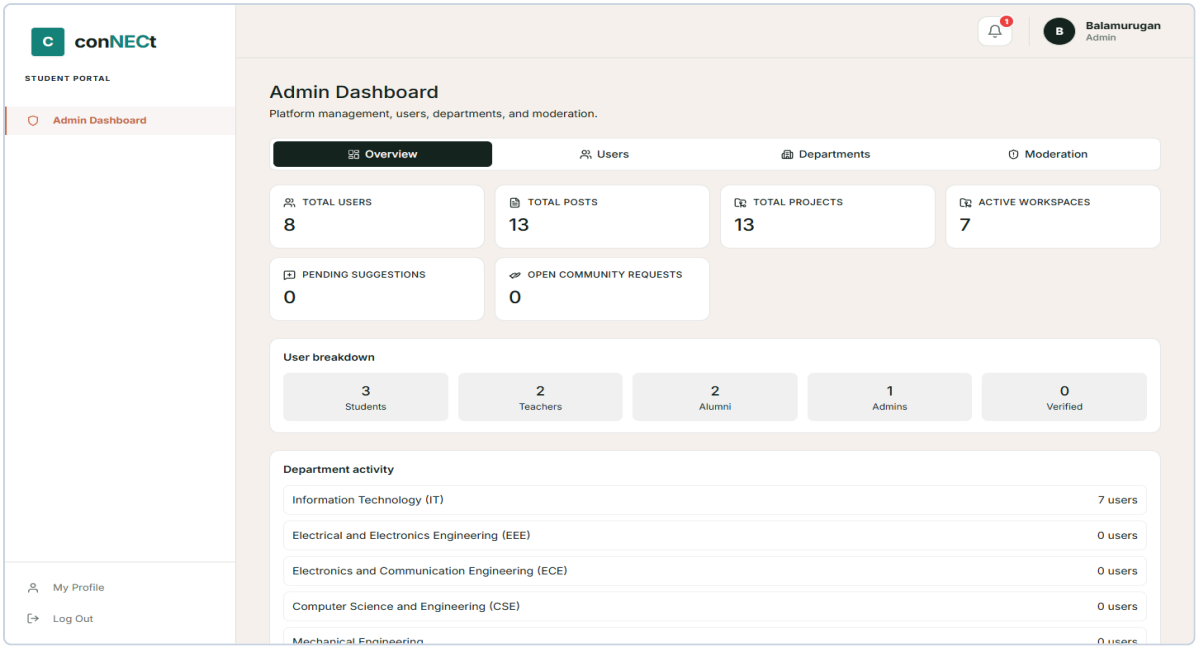



**Fig 8.14 - Administrator Overview with Platform Analytics & Metrics**

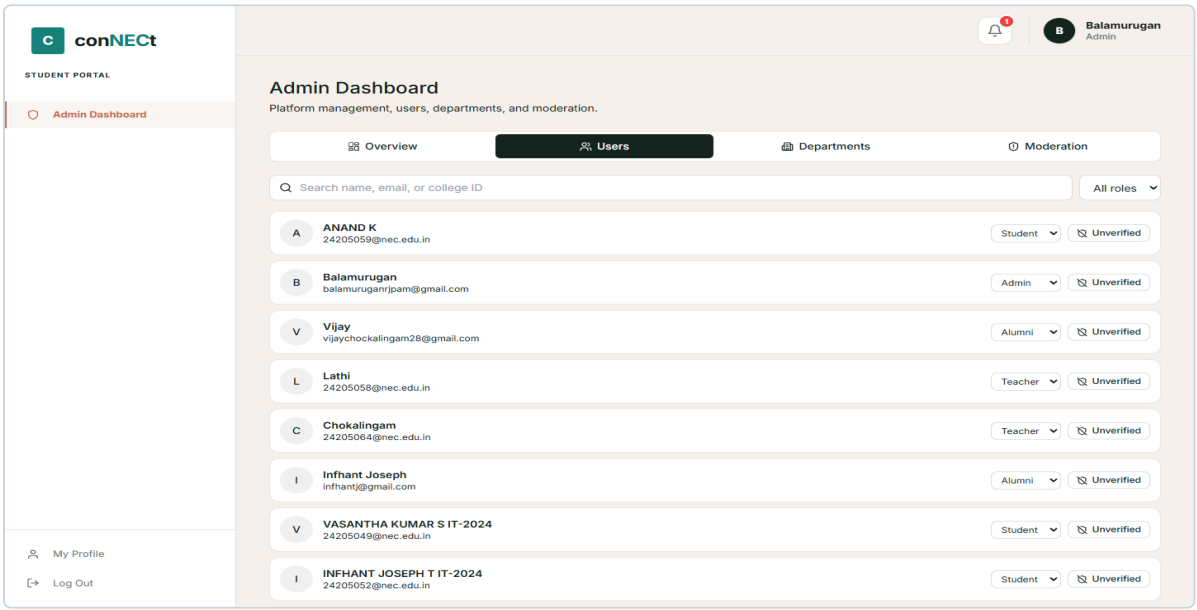



**Fig 8.15 - Administrator User Directory with Role and Verification Controls**

The Admin Dashboard equips college authorities with platform governance utilities. Administrators can monitor real-time counts of active users, projects, and department activities. The Users tab enables administrators to filter accounts by role, verify newly registered alumni credentials, manage permissions, and maintain college community standards.

# CHAPTER 9

# CONCLUSION

In conclusion, the **conNECt Campus Collaboration & Networking Portal** delivers a complete, secure, and modern digital platform that unifies students, faculty, alumni, and administrators of National Engineering College into a single collaborative ecosystem. By combining institutional academic management with rich social and professional networking, the system overcomes the limitations of traditional, disconnected college websites.

The platform successfully provides role-based authentication, interactive campus timelines, structured academic courseware spaces, multidisciplinary project incubators, formal mentorship programs, and real-time WebSocket communications. It ensures data protection through salted bcrypt password encryption and JWT authorization middleware, while MongoDB delivers the scalability required to support thousands of active campus interactions.

Through conNECt, students gain access to industry mentors and peer study partners, faculty members manage course materials with ease, alumni maintain active ties with their institution, and administrators exercise governance over the college digital infrastructure. The responsive, accessible user interface guarantees an exceptional user experience on both desktop and mobile devices.

Future enhancements will focus on integrating automated AI-based code review assistants for capstone projects, automated college timetable schedule synchronization, and push notifications for mobile devices. Overall, conNECt serves as a robust, professional, and scalable digital network that elevates academic excellence and campus networking at National Engineering College.